AE1.1 - CRUD con Ficheros

CRUD de videojuegos con ficheros de texto y binarios en GameHub

Elemento	Descripción
Módulo	Acceso a Datos
Curso	2º DAM
Unidad de trabajo	UT1. Manejo de ficheros en Java
Proyecto transversal	GameHub
Tipo de actividad	Actividad evaluable individual.

1. Contexto de la actividad

En esta actividad vas a desarrollar una primera versión funcional de GameHub, una aplicación de consola para gestionar información de videojuegos utilizando persistencia en ficheros.

La aplicación trabajará con datos cargados en memoria y permitirá decidir cuándo guardar la información a ficheros.

2. Descripción general

Debes crear una aplicación Java de consola que permita gestionar un catálogo de videojuegos mediante un CRUD completo.

El programa deberá mantener una colección de objetos Videojuego en memoria durante la ejecución. Todas las operaciones de alta, consulta, modificación y borrado se realizarán sobre esa colección.

Además, el menú deberá incluir opciones para guardar la información en un fichero binario secuencial y a un fichero de texto separado por el carácter dos puntos (:).

3. Modelo de datos obligatorio

Debes crear una **clase Videojuego**. Cada videojuego deberá almacenar, como mínimo, la siguiente información:

Campo	Tipo recomendado	Descripción
id	int	Identificador único del videojuego.
titulo	String	Nombre del videojuego.
plataforma	String	Plataforma: PC, PS5, Switch, Xbox, etc.
genero	String	Género del videojuego.
horasJugadas	double	Número de horas jugadas.
finalizado	boolean	Indica si el juego está terminado.

La clase deberá incluir atributos privados, constructor, Getters y setters cuando sean necesarios, Método toString(), Implementación de Serializable para poder guardar objetos en fichero binario.

4. Requisitos funcionales

4.1. Menú principal

La aplicación deberá mostrar un menú similar al siguiente:

```
==============================
        GAMEHUB - UT1
==============================
1. Cargar datos desde fichero de texto
2. Cargar datos desde fichero binario
3. Añadir videojuego
4. Buscar videojuego por ID
5. Mostrar todos los videojuegos
6. Modificar videojuego
7. Eliminar videojuego
8. Guardar datos en fichero de texto
9. Guardar datos en fichero binario

0. Salir
Seleccione una opción:
```

El menú debe repetirse hasta que el usuario seleccione la opción 0.

4.2. Cargar datos desde fichero de texto

El programa deberá leer los videojuegos desde el fichero **data/videojuegos.txt**.

Cada línea representará un videojuego con el siguiente formato:

—> id:titulo:plataforma:genero:horasJugadas:finalizado

Ejemplo: 1:Hollow Knight:PC:Metroidvania:45.5:true

4.3. Cargar datos desde fichero binario

El programa deberá permitir cargar los videojuegos desde el fichero **data/videojuegos.dat**.

La carga deberá reconstruir la colección de videojuegos en memoria.

Puede realizarse mediante serialización de objetos con **ObjectInputStream/ObjectOutputStream** o mediante fichero binario secuencial con DataInputStream/DataOutputStream.

4.4. Añadir videojuego

El usuario deberá introducir los datos de un nuevo videojuego. El programa deberá validar que el id no exista ya en memoria, que el título no esté vacío y que las horas jugadas no sean negativas. Si el id ya existe, se mostrará un mensaje de error y no se añadirá el videojuego.

4.5. Buscar videojuego por ID

El usuario introducirá un id. El programa buscará el videojuego en la colección en memoria. Si existe, se mostrará toda su información. Si no existe, se mostrará un mensaje indicando que no se ha encontrado.

4.6. Mostrar todos los videojuegos

El programa mostrará todos los videojuegos almacenados actualmente en memoria. Si no hay videojuegos cargados, se mostrará un mensaje informativo.

4.7. Modificar videojuego

El usuario introducirá el id del videojuego que desea modificar. Si existe, el programa permitirá modificar título, plataforma, género, horas jugadas y estado de finalización. No se recomienda modificar el id. Si el videojuego no existe, se mostrará un mensaje de error.

4.8. Eliminar videojuego

El usuario introducirá el id del videojuego que desea eliminar. Antes de eliminarlo, el programa deberá pedir confirmación. Si el usuario confirma, el videojuego se eliminará de la colección en memoria.

4.9. Guardar datos en fichero de texto

El programa deberá guardar todos los videojuegos que están actualmente en memoria en data/videojuegos.txt. El fichero deberá sobrescribirse completamente con la información actual. Cada videojuego deberá ocupar una línea con el formato **id:titulo:plataforma:genero:horasJugadas:finalizado**.

4.10. Guardar datos en fichero binario

El programa deberá guardar todos los videojuegos que están actualmente en memoria en **data/videojuegos.dat**. Este fichero debe permitir recuperar posteriormente la información. Se valorará el uso correcto de ObjectOutputStream/ ObjectInputStream o de DataOutputStream/DataInputStream.

4.11. Salida del programa

Cuando el usuario seleccione la opción 0, el programa no debe cerrarse inmediatamente. Primero deberá preguntar: Hay datos en memoria. ¿Desea actualizar los ficheros antes de salir? (s/n). Si el usuario responde s, el programa deberá guardar la información actual en el fichero de texto principal y en el fichero binario principal.Si responde n, el programa saldrá sin actualizar los ficheros.

5. Ficheros obligatorios

Fichero	Finalidad
data/videojuegos.txt	Fichero principal de texto para guardar/cargar datos.
data/videojuegos.dat	Fichero principal binario para guardar/cargar datos.

6. Requisitos de diseño: MVC Opcional.

La aplicación debe implementar el patrón Modelo - Vista - Controlador. La estructura propuesta podría ser:

```
src/
 └── main/
     └── java/
         └── com/gamehub/
             ├── model/
             │   └── Videojuego.java
             ├── view/
             │   └── GameHubView.java
             ├── controller/
             │   └── GameHubController.java
             ├── repository/
             │   ├── VideojuegoRepository.java
             │   ├── TextVideojuegoRepository.java
             │   └── BinaryVideojuegoRepository.java
             ├── service/
             │   └── VideojuegoService.java
             ├── exception/
             │   └── GameHubFileException.java
             └── Main.java
```

Parte	Responsabilidad
Modelo	Representa los datos de la aplicación. No debe pedir datos por teclado ni escribir directamente en pantalla.
Vista	Muestra el menú, pide datos por teclado, muestra mensajes y solicita confirmaciones.
Controlador	Recibe la opción elegida, llama al servicio correspondiente y coordina el flujo del programa.
Servicio	Contiene la lógica de negocio: validaciones, CRUD y colección en memoria.
Repositorios	Contienen la lógica de acceso a ficheros: leer, escribir, exportar y gestionar errores de entrada/salida.

7. Requisitos técnicos

- El programa deberá mantener los videojuegos en memoria durante la ejecución, por ejemplo mediante ArrayList<Videojuego>.
- Todas las operaciones CRUD se realizarán sobre la colección en memoria.
- Debe utilizarse try-with-resources siempre que se abran flujos de lectura o escritura.
- Deben controlarse errores como fichero inexistente, error de lectura, error de escritura, formato incorrecto, entrada numérica incorrecta e intento de modificar o eliminar un videojuego inexistente.
- Se recomienda crear una excepción personalizada, por ejemplo GameHubFileException, para encapsular errores de fichero.

8. Restricciones

- No se permite resolver toda la aplicación dentro de Main.
- No se permite mezclar código de menú con código de escritura en ficheros.
- No se permite guardar datos directamente desde la vista.
- No se permite acceder al teclado desde el modelo o los repositorios.
- No se permite usar bases de datos.
- No se permite usar librerías externas para la persistencia.
- No se permite ignorar excepciones con bloques catch vacíos.

9. Entrega

El alumnado deberá entregar:

- Proyecto Java completo.
- README.md. Capturas de ejecución con su explicación.

El README debe incluir una descripción breve del proyecto, explicación de la estructura MVC, formato de los ficheros, instrucciones de ejecución, ejemplo de datos y decisiones tomadas sobre el guardado binario.